

CSCE 735 PARALLEL COMPUTING · MODULE 06 · LECTURE NOTES

Decomposition and Load Balancing

Dividing the work so that nobody waits.

MODULE OUTCOMES

- Choose a decomposition strategy for a problem.
- Explain granularity and its trade-off.
- Implement work stealing and explain why it balances.
- Recognise load imbalance in a profile.
- Apply the standard parallel patterns.

1 Decomposition

Strategy	Divides by	Suits	Scales with
Data (domain) decomposition	Regions of the data — rows, blocks, tiles, particles.	Arrays, grids, images, meshes. The dominant strategy in numerical work.	The size of the data.
Task (functional) decomposition	Different operations performed concurrently.	Heterogeneous work, pipelines, programs with distinct phases.	The number of distinct tasks — which is usually small and fixed, so this does not scale . A frequently missed limitation.
Recursive decomposition	Divide and conquer — split, recurse, combine.	Trees, sorts, spatial structures, anything already recursive.	The depth of the recursion, which grows with the data.
Pipeline	Stages of a sequential process, with data flowing through.	Streaming workloads where each item passes through the same stages.	The number of stages — and throughput is bounded by the slowest stage , so balancing the stages is the whole problem.

GRANULARITY IS THE CENTRAL TRADE-OFF

Too coarse and load imbalance dominates. With four chunks distributed over four cores, a single chunk that takes twice as long leaves three cores idle for half the phase, and no scheduler can recover — there is nothing left to redistribute.

Too fine and scheduling overhead dominates the useful work. A task that takes one microsecond does not justify a queue push, a queue pop, and the cache traffic between them.

The rule of thumb: create enough chunks that imbalance averages out — commonly several times the thread count, so that a slow chunk is absorbed by its neighbours — **while keeping each chunk large enough to amortise the scheduling cost**, which means tens of microseconds or more for a typical task runtime.

And measure. The correct granularity depends on the per-item cost and the runtime's overhead, both of which are specific to your situation and both of which are easy to determine empirically by sweeping the chunk size.

2 Load balancing

A BARRIER COSTS THE SLOWEST THREAD'S TIME, N TIMES OVER

At a barrier, every thread waits for the last one to arrive. So the wall-clock cost of a parallel phase is the *maximum* thread's time, not the average.

If one thread's share takes twice as long as the others', every other thread idles for half the phase. On 32 threads that is 31 cores doing nothing for half the time — a loss of nearly 50% of the machine, caused by one thread.

And it is invisible in total CPU time, because the waiting threads are either spinning (which looks like full utilisation) or sleeping (which looks like idle time with no attributable cause). Neither tells you which thread was slow or why.

So load imbalance goes directly into Amdahl's serial fraction (Module 01), and in practice it is frequently the largest single contributor to it — larger than the genuinely sequential code it is usually blamed on.

	Static balancing	Dynamic balancing
Assignment	Decided before the computation begins, in equal shares.	Threads take work as they become free.
Runtime overhead	None at all.	A queue operation per chunk, plus the cache traffic it causes.
Locality	Good — each thread owns a contiguous region and touches it repeatedly, which also places NUMA pages correctly (Module 02).	Worse — a thread may process chunks scattered through memory, and which chunks it gets varies between runs.

	Static balancing	Dynamic balancing
Requires	The per-item cost to be known and roughly uniform.	The work to be divisible into chunks at runtime.
Under skew	Fails completely.	Balances automatically.

Each worker owns a double-ended queue (deque).

OWN work: push and pop at the BOTTOM -- LIFO

- > most recently pushed task popped first: its data is in cache
- > and it is deepest in the recursion, so it is small

STEALING: thieves take from the TOP -- FIFO

- > the oldest task, which is the LARGEST remaining subtree
- > so one steal transfers a lot of work
- > thief and victim touch opposite ends: no contention

Idle worker: pick a random victim, try to steal, repeat.

THE LIFO-OWN / FIFO-STEAL ASYMMETRY IS THE WHOLE DESIGN

Popping your own work LIFO gives cache locality. The most recently pushed task operates on data you have just touched, so it is still resident. A FIFO local order would touch the oldest data first, which is the coldest.

Stealing FIFO transfers large chunks. In a recursive decomposition, the oldest task on the deque is the one highest in the recursion tree — which represents the *largest* remaining subtree. So a single steal transfers a great deal of work, and steals are consequently rare.

And thief and victim access opposite ends of the deque, so in the common case where the deque is not nearly empty, the two never touch the same memory and no synchronisation is needed at all. Only the nearly-empty case requires a CAS.

Three desirable properties fall out of one data structure choice, which is why this design is used essentially universally and why it is worth studying as a piece of engineering rather than just learning the interface.

WORK STEALING IS PROVABLY NEAR-OPTIMAL

Blumofe and Leiserson proved that randomised work stealing completes a computation in expected time within a constant factor of the optimal schedule, with provably bounded space usage and bounded communication. This is an unusually strong result for a scheduling heuristic.

It requires no advance knowledge of task costs, which is what makes it practical — in real workloads those costs depend on the data and are generally unknowable before the work is done.

It adapts to whatever else the machine is doing. If another process takes a core away, the remaining workers simply steal more, with no reconfiguration.

And it is what Cilk, Intel TBB, OpenMP tasks, Go's goroutine scheduler, Rust's rayon, Java's ForkJoinPool, and .NET's task scheduler all use. The convergence of independent designs on the same algorithm is not coincidence.

3 Parallel patterns

Pattern	Shape	Parallel structure
Map	Apply a function independently to every element.	Trivially parallel. $O(1)$ depth.
Reduce	Combine all elements with an associative binary operator.	A tree — $O(\log n)$ depth, $O(n)$ work. Requires associativity; see below.
Scan (prefix sum)	Produce all prefix reductions — output i is the reduction of inputs $0..i$.	Appears inherently sequential and is not — $O(\log n)$ depth. Module 11, and the most surprising result in the course.
Stencil	Each output computed from a neighbourhood of inputs.	Parallel over the interior; boundaries require a halo exchange (Module 10).
Gather / scatter	Indexed read / indexed write.	Gather is parallel; scatter requires conflict handling , since two sources may write the same destination.
Partition / filter	Split elements by a predicate.	A scan over the predicate flags gives each surviving element its output position.

Recognising the pattern gives you the parallel algorithm, its complexity, and usually a library implementation. Most parallel code is a composition of these six, and pattern recognition is the most practically useful skill in this module.

ASSOCIATIVITY IS WHAT MAKES REDUCTION PARALLEL

A reduction can be computed as a tree only if the operator is associative — that is, only if $(a \oplus b) \oplus c = a \oplus (b \oplus c)$. The tree evaluates the combinations in a different order from the sequential loop, and associativity is exactly the guarantee that the order does not matter.

Integer addition is associative. Floating-point addition is not. Rounding occurs after each operation, so $(a+b)+c$ and $a+(b+c)$ can differ in the last bits — and for values of widely differing magnitude they can differ substantially.

So a parallel floating-point reduction gives a different answer from the serial one, and a *different* answer for different thread counts, and potentially a different answer between runs if the schedule varies (CSCE 649 Module 13 met the same problem from the determinism side).

This is not a bug and it must be expected. If bitwise reproducibility is required — for regression testing, for debugging, or for a scientific result that must be replicable — then the reduction order must be fixed explicitly, which costs performance. **Decide which you need**, rather than discovering the variation later and treating it as a defect.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Blumofe & Leiserson — Scheduling Multithreaded Computations by Work Stealing (free)	The §2 result, with the proof. The deque design is in the Cilk papers from the same group.
McCool, Robison & Reinders — Structured Parallel Programming	The pattern catalogue of §3, systematically. Library copy; the pattern list is widely reproduced.
Intel TBB documentation (free)	A production work-stealing runtime, documented, including the granularity guidance in §1.
Stanford CS149 — work distribution and scheduling lectures (free)	Decomposition, granularity, and load balancing with worked measurements.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Take a loop with uniform cost and one with highly variable cost. Measure static and dynamic scheduling on both and explain the four results.
- 2 Sweep the chunk size over three orders of magnitude for a dynamic schedule and plot the time. Identify both failure modes.
- 3 **Measure load imbalance directly:** instrument each thread's busy time within a phase and plot the distribution.
- 4 Construct a workload where one thread takes twice as long and measure the wasted core-seconds.
- 5 Implement a work-stealing deque with LIFO local and FIFO steal operations.
- 6 Measure the steal rate for a balanced and a skewed workload, and confirm steals are rare in the balanced case.
- 7 Implement your own LIFO-steal variant and measure how much worse it is. Explain the result.
- 8 Implement map, reduce, and scan over an array and measure the scaling of each.
- 9 **Demonstrate float non-associativity:** sum an array of mixed magnitudes serially and with 2, 4, and 8 threads. Report all the answers.
- 10 Implement a deterministic parallel reduction with a fixed combination order and measure its cost against the unordered version.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name four decomposition strategies and say what each scales with.
- 2 Why does task decomposition not scale?
- 3 State the granularity trade-off and the rule of thumb.
- 4 Why does a barrier cost the slowest thread's time n times, and why is it invisible in CPU time?
- 5 Compare static and dynamic balancing on five axes.
- 6 Explain the LIFO-own / FIFO-steal design and the three properties it provides.
- 7 What did Blumofe and Leiserson prove, and why does it matter practically?
- 8 Name six parallel patterns and the parallel structure of each.
- 9 Why is a parallel float reduction not reproducible, and what are your options?